

copy_of_LLM_Assignment_1_(Coding)

April 30, 2026

1 LLM Assignment 1: Representation Surgery (Coding)

IMPORTANT: MAKE A COPY OF THIS NOTEBOOK AND WORK IN THERE.

Welcome to the coding part of the Representation Surgery question in Assignment 1. The goal of this portion is to give you a deeper intuition for how representation surgery works and its strengths/limitations via hands-on implementations.

It consists of 2 parts: 1. Implementing linear concept erasure as described in the assignment, and applying it to a synthetic dataset. 2. Applying concept erasure to erase the concept of “gender” from the BERT representations of a dataset of biographies.

In this assignment, you’ll have a series of tasks. For each task you will need to write some code and produce plots and/or accuracy results. You will need to export the notebook as a pdf and attach it to your submission. Please make sure all deliverables for each task (e.g., plots and accuracy results) are clearly visible, as we will be grading you on these outputs.

```
[ ]: !git clone https://github.com/rycolab/llm-representation-surgery.git
```

fatal: destination path 'llm-representation-surgery' already exists and is not an empty directory.

```
[ ]: # These are all the imports you should need
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.cross_decomposition import CCA
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from typing import Tuple
np.random.seed(42)
```

1.1 0. Constructing a synthetic dataset

First, we’ll give you a synthetic dataset encoding a concept, i.e., the target labels.

```
[ ]: n, d, k = 2048, 128, 2
X, Z = make_classification(
```

```

n_samples=n,
n_informative=k,
n_features=d,
n_classes=k,
random_state=1,
)
print(f"Shape of X: {X.shape}, Shape of Z: {Z.shape}")

```

Shape of X: (2048, 128), Shape of Z: (2048,)

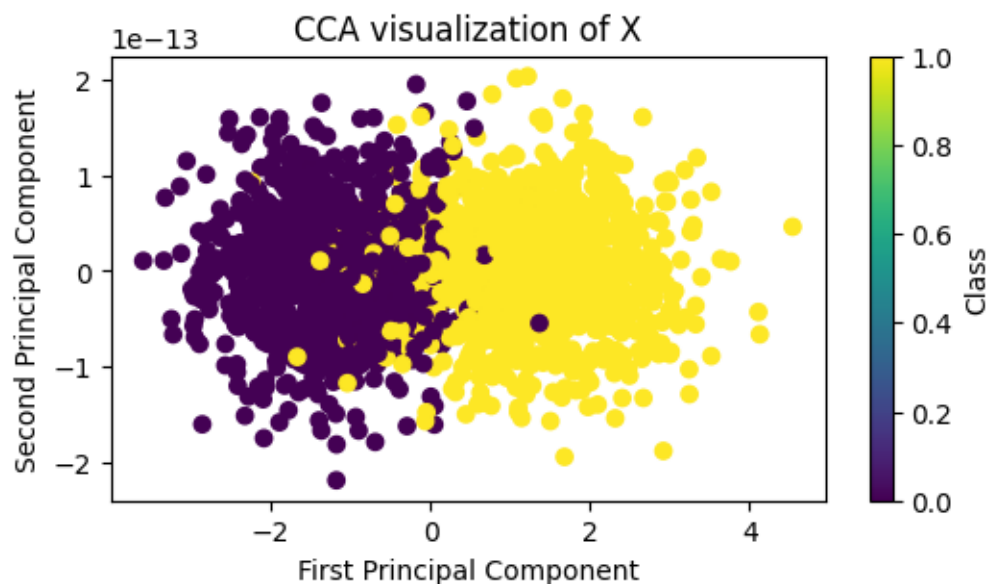
What does this dataset look like? Visualizing 128-dimensional data is hard, so we'll reduce the dimensionality to 2 using CCA and plot the result.

```

[ ]: def plot_cca(X, Z):
    """
    Use Canonical Component Analysis to visualize the dataset X and label Z.
    """
    Z_one_hot = np.zeros((len(Z), len(set(Z))))
    Z_one_hot[np.arange(len(Z)), Z] = 1
    cca = CCA(n_components=2)
    X_cca = cca.fit_transform(X, y=Z_one_hot)[0]
    plt.figure(figsize=(6, 3))
    plt.scatter(X_cca[:, 0], X_cca[:, 1], c=Z, cmap='viridis')
    plt.colorbar(label='Class')
    plt.xlabel('First Principal Component')
    plt.ylabel('Second Principal Component')
    plt.title('CCA visualization of X')
    plt.show()

plot_cca(X, Z)

```



As you can see, in these two components, the two concepts are fairly separable.

1.1.1 Task 0.1: Implement a logistic regression classifier predict the concept Z from X .

FOR THE SUBMISSION: Print the accuracy your classifier scores on the data.

```
[ ]: def logreg(X: np.ndarray, Z: np.ndarray) -> float:
    """
    Train a logistic regression classifier to predict concept Z from X.

    Args:
        X - the data
        Z - the concept/label

    Return:
        Accuracy in predicting Z from X.
    """
    clf = LogisticRegression(max_iter=1000)
    clf.fit(X, Z)
    return clf.score(X, Z)

print(f"Accuracy before erasure: {logreg(X, Z):.3f}")
```

Accuracy before erasure: 0.941

1.2 1. Implementing linear concept erasure

1.2.1 Task 1.1: Validate that the resulting whitening and unwhitening matrices satisfy their definition and goals.

That is, verify (a) $WW^\top = I$ and (b) $\text{cov}(XW) = I$.

FOR THE SUBMISSION: Include your plots showing the whitening matrices.

```
[ ]: def compute_W(X: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
    """
    Compute the whitening and unwhitening matrix for the data using the ZCA_
    ↪ whitening method
    (see Appendix A of https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf for more details).
    We give this to you as a numerically stable implementation.
    """
    sigma = np.cov(X.T)

    L, V = np.linalg.eigh(sigma)
```

```

# Assuming PSD; account for numerical error
L = L.clip(0.0)

# Threshold used by torch.linalg.pinv
mask = L > (L[-1] * sigma.shape[-1] * np.finfo(float).eps)

with np.errstate(divide='ignore'):
    W = V * np.where(mask, 1/np.sqrt(L), 0.0) @ V.T
    W_inv = V * np.where(mask, np.sqrt(L), 0.0) @ V.T

return W, W_inv

def heatmap(arr, title):
    plt.figure(figsize=(10, 8))
    sns.heatmap(arr, cmap='viridis')
    plt.title(title)
    plt.tight_layout()
    plt.show()

W, W_inv = compute_W(X)

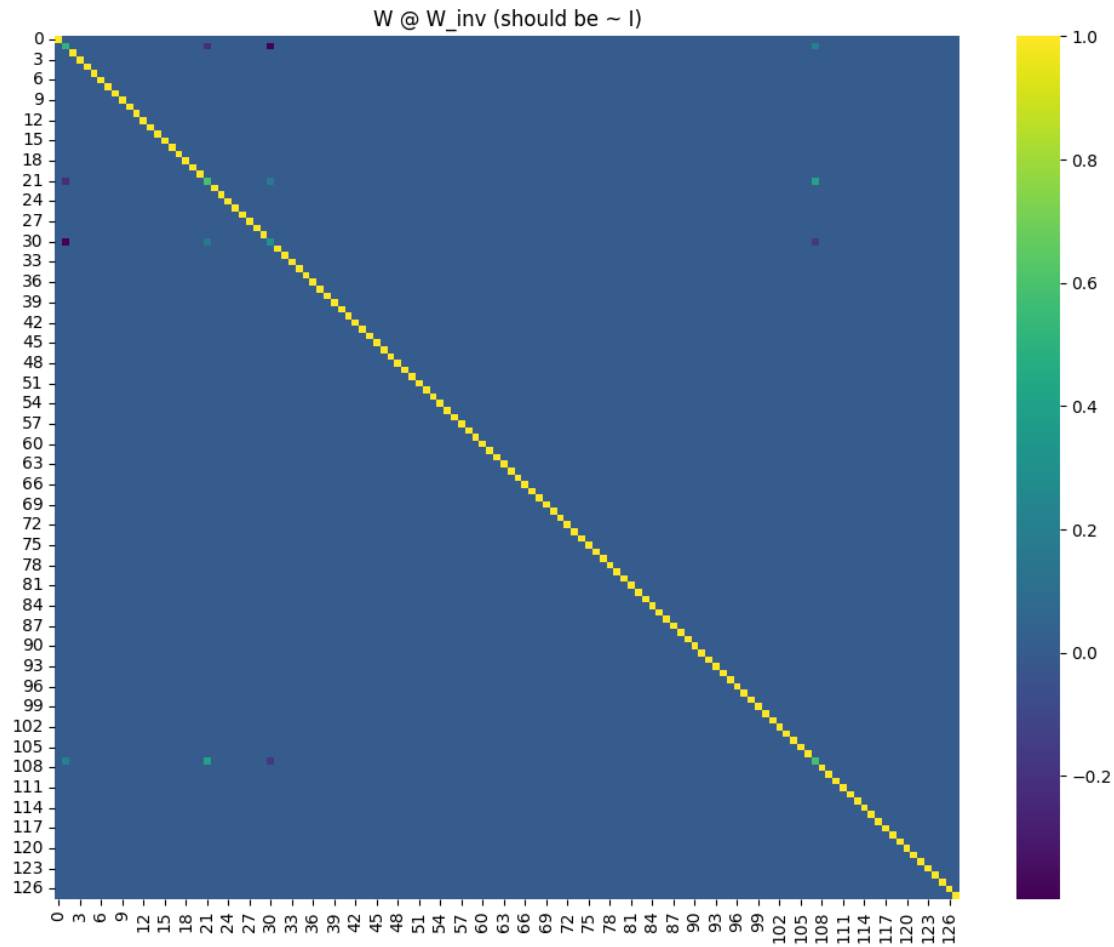
```

```

[ ]: def verify_W_W_inv(W, W_inv):
    """
    Plot a heatmap of  $W @ W_{inv}$  to verify it satisfies the definition of
    ↪whitening.
    """
    heatmap(W @ W_inv, title="W @ W_inv (should be ~ I)")

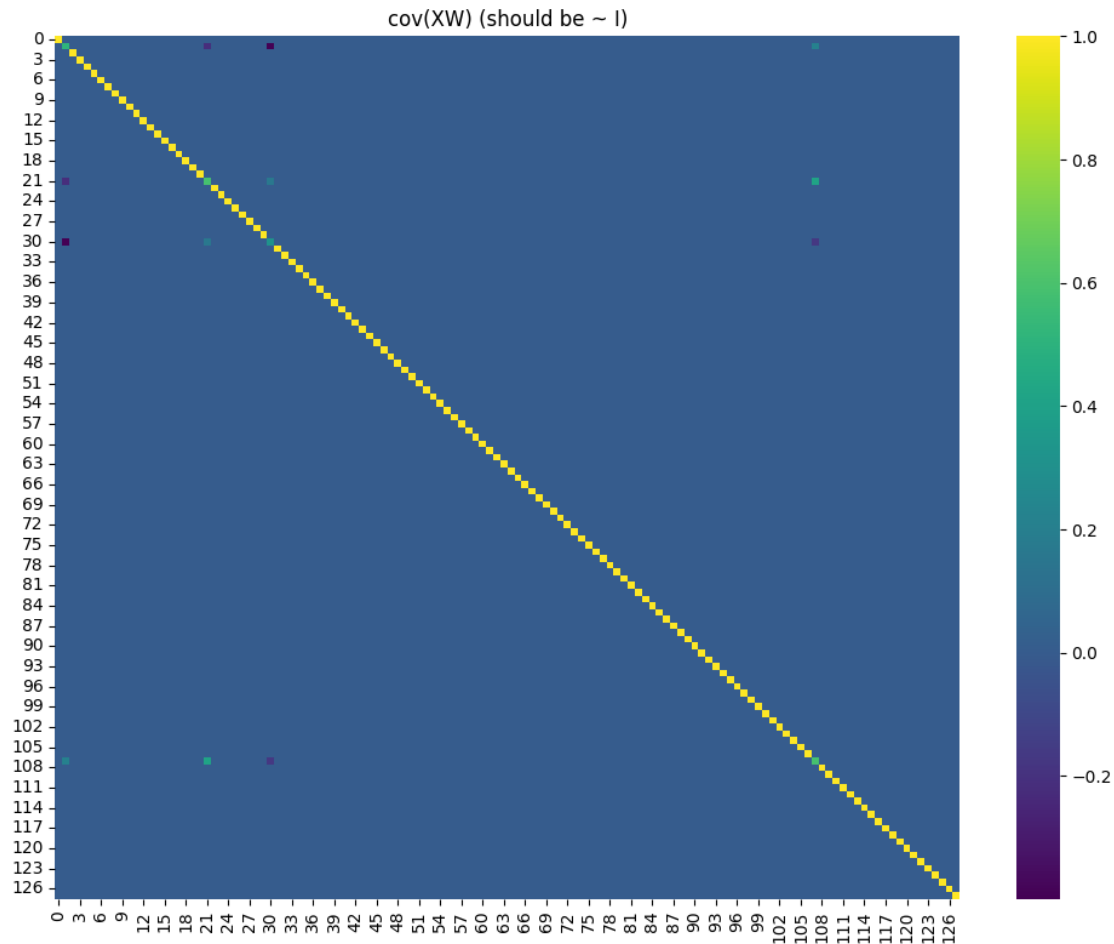
verify_W_W_inv(W, W_inv)

```



```
[ ]: def verify_cov_XW(X, W):
    """
    Plot a heatmap of the covariance of XW to verify it satisfies the
    definition of whitening.
    """
    cov_XW = np.cov((X @ W).T)
    heatmap(cov_XW, title="cov(XW) (should be ~ I)")

verify_cov_XW(X, W)
```



1.2.2 Task 1.2: Implement linear concept erasure

Use the equations derived from the written part of the assignment.

FOR THE WRITEUP: Include your plot and accuracy of the embeddings before erasure and after erasure.

```
[ ]: def erase(X: np.ndarray, Z: np.ndarray) -> np.ndarray:
    """
    Implement linear concept erasure.

    Args:
        X - the data, shape (n, d)
        Z - the concept/label, shape (n,)

    Return:
        The data with the concept erased. Shape (n, d).
    """
```

```

n = X.shape[0]
k = len(np.unique(Z))

Z_one_hot = np.zeros((n, k))
Z_one_hot[np.arange(n), Z] = 1

mean_X = X.mean(axis=0)
X_centered = X - mean_X
Z_centered = Z_one_hot - Z_one_hot.mean(axis=0)

Sigma_XZ = (X_centered.T @ Z_centered) / n

W, W_inv = compute_W(X)

WSigma = W @ Sigma_XZ
Pi = WSigma @ np.linalg.pinv(WSigma)

return X - X_centered @ W @ Pi @ W_inv

erased_X = erase(X, Z)

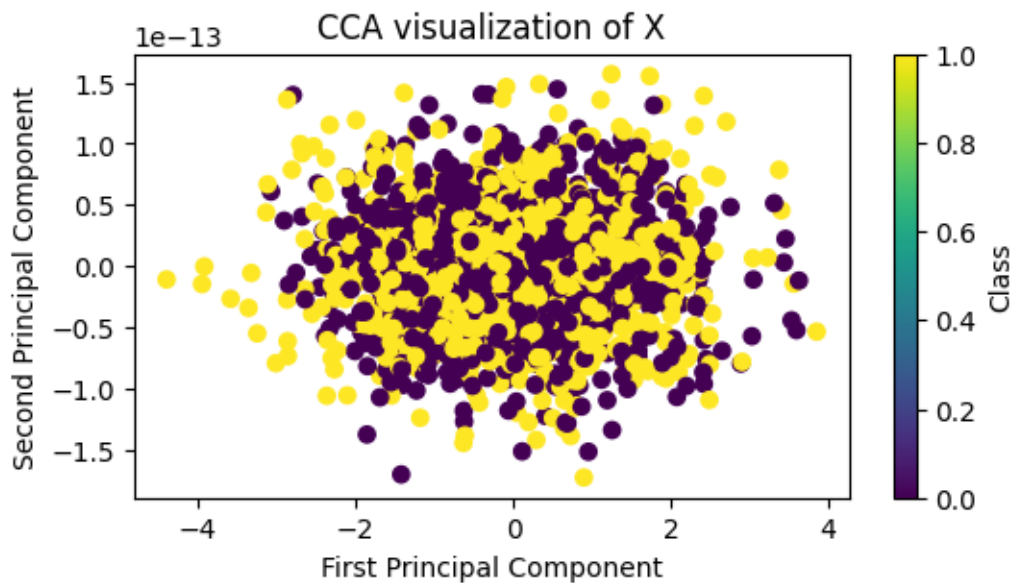
```

Now, let's validate that the concept has been erased.

```

[ ]: plot_cca(erased_X, Z)
print(f"Accuracy after erasure: {logreg(erased_X, Z)}")

```



Accuracy after erasure: 0.501953125

1.3 2. Erasing gender bias

In this portion, we'll apply the same method to a real dataset of word embeddings of biography data. Your goal is to (a) construct sentence embeddings and (b) erase the gender concept from these embeddings. Use the `hard_text_untokenized` column for your embeddings.

```
[ ]: df = pd.read_csv("llm-representation-surgery/data/biographies.csv")
df.head()
```

```
[ ]:      g          p          text \
0  f  psychologist  Brandi Smith is a registered psychologist in t...
1  m  software_engineer  Duarte Nunes is a Software Engineer working on...
2  f  software_engineer  Lauren Ellsworth is a Senior Software Engineer...
3  f          professor  Gillian McCann is an Associate Professor in th...
4  m          professor  BIOGRAPHY: Jason Del Gandio (PhD) is an Assist...

      start          hard_text \
0    139  Ms. Smith is experienced in providing psychoth...
1     56  He has a background in concurrent programming ...
2    139  In her years of development , Lauren has contr...
3    124  She holds a doctorate in Religious Studies in ...
4    113  His area of expertise is the theory and practi...

          hard_text_untokenized \
0  Ms. Smith is experienced in providing psychoth...
1  He has a background in concurrent programming,...
2  In her years of development, Lauren has contri...
3  She holds a doctorate in Religious Studies in ...
4  His area of expertise is the theory and practi...

          text_without_gender
0  _ . _ is experienced in providing psychotherapy...
1  _ has a background in concurrent programming, ...
2  In _ years of development, _ has contributed t...
3  _ holds a doctorate in Religious Studies in th...
4  _ area of expertise is the theory and practice...
```

1.3.1 Task 2.1: Compute embeddings and use the erase function defined above to erase the gender concept from the embeddings.

FOR THE FOR THE SUBMISSION: Include: (a) CCA plot of the embeddings before erasure, (b) CCA plot of the embeddings after erasure, (c) logistic regression accuracy before erasure, and (d) logistic regression accuracy after erasure.

```
[ ]: from transformers import AutoModel, AutoTokenizer, logging
import torch
import warnings
import os
```

```

# Suppress warnings and logging messages
warnings.filterwarnings("ignore")
logging.set_verbosity_error()
os.environ["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"

def extract_gender_labels(df: pd.DataFrame) -> np.ndarray:
    """
    Extract the gender labels from the dataframe.
    """
    return LabelEncoder().fit_transform(df["g"])

def compute_embeddings(df: pd.DataFrame) -> np.ndarray:
    """
    Compute the embeddings using a pretrained model for the
    ↪ "hard_text_untokenized" column of the given dataframe.
    Feel free to experiment with different models; a good starting point is the
    ↪ classic BERT model, "google-bert/bert-base-uncased".
    Also feel free to use any libraries which would be helpful, such as
    ↪ `transformers` or `sentence_transformers`.
    """
    model_name = "google-bert/bert-base-uncased"
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name)
    device = (
        "cuda" if torch.cuda.is_available()
        else ("mps" if torch.backends.mps.is_available() else "cpu")
    )
    model.to(device).eval()

    texts = df["hard_text_untokenized"].astype(str).tolist()
    batch_size = 32
    embeddings = []
    with torch.no_grad():
        for i in range(0, len(texts), batch_size):
            batch = texts[i : i + batch_size]
            enc = tokenizer(
                batch,
                padding=True,
                truncation=True,
                max_length=128,
                return_tensors="pt",
            ).to(device)
            out = model(**enc).last_hidden_state # (B, T, H)
            mask = enc["attention_mask"].unsqueeze(-1) # (B, T, 1)
            pooled = (out * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
            embeddings.append(pooled.cpu().numpy())

```

```

    return np.vstack(embeddings)

def erase_gender(embeddings: np.ndarray, gender_labels: np.ndarray) -> np.
    ndarray:
    """
    This function should do 3 things:
    1. Erase the gender concept from the embeddings and return these new
    embeddings.
    2. Visualize the embeddings before erasure and print the accuracy
    before erasure.
    3. Visualize the embeddings after erasure and print the accuracy after
    erasure.

    Args:
        embeddings: The embeddings to erase the gender concept from.
        gender_labels: The gender labels for the embeddings.

    Returns:
        The embeddings with the gender concept erased.
    """
    print("--- Before erasure ---")
    plot_cca(embeddings, gender_labels)
    print(
        f"Linear accuracy before erasure: "
        f"{logreg(embeddings, gender_labels):.3f}"
    )

    erased = erase(embeddings, gender_labels)

    print("--- After erasure ---")
    plot_cca(erased, gender_labels)
    print(
        f"Linear accuracy after erasure: "
        f"{logreg(erased, gender_labels):.3f}"
    )
    return erased

gender_labels = extract_gender_labels(df)

print("Gender label distribution:")
print(df["g"].value_counts())
print(f"\nMajority class proportion: {df['g'].value_counts().max() / len(df):.
    3f}")

embeddings = compute_embeddings(df)
embeddings_erased = erase_gender(embeddings, gender_labels)

```

```

Z_onehot = np.zeros((len(gender_labels), len(set(gender_labels))))
Z_onehot[np.arange(len(gender_labels)), gender_labels] = 1
Sigma_XZ = np.cov(embeddings_erased.T, Z_onehot.T)[:embeddings_erased.shape[1],
↳ embeddings_erased.shape[1]:]
print(f"Sanity check -> max |cov(erased, Z)|: {np.max(np.abs(Sigma_XZ)).2e}")

```

Gender label distribution:

```

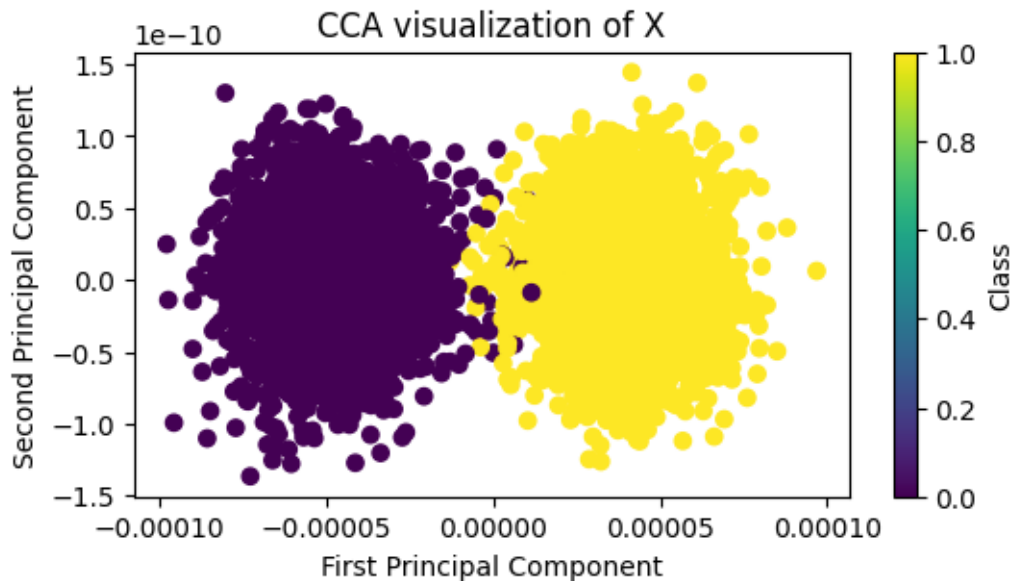
g
m    5467
f    4533
Name: count, dtype: int64

```

Majority class proportion: 0.547

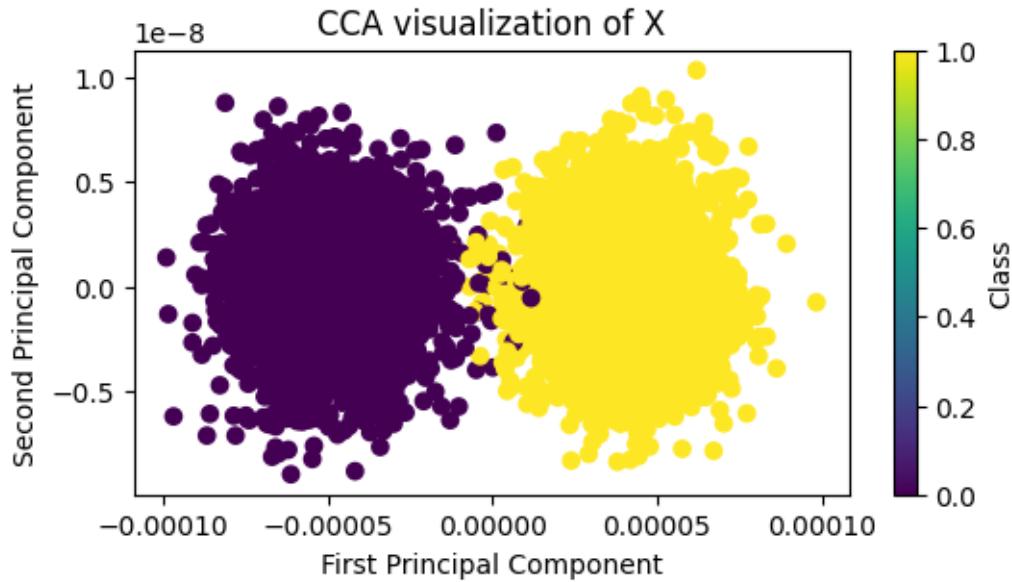
Loading weights: 0% | 0/199 [00:00<?, ?it/s]

--- Before erasure ---



Linear accuracy before erasure: 0.999

--- After erasure ---



Linear accuracy after erasure: 0.576
 Sanity check -> $\max |\text{cov}(\text{erased}, Z)|$: $7.14\text{e-}04$

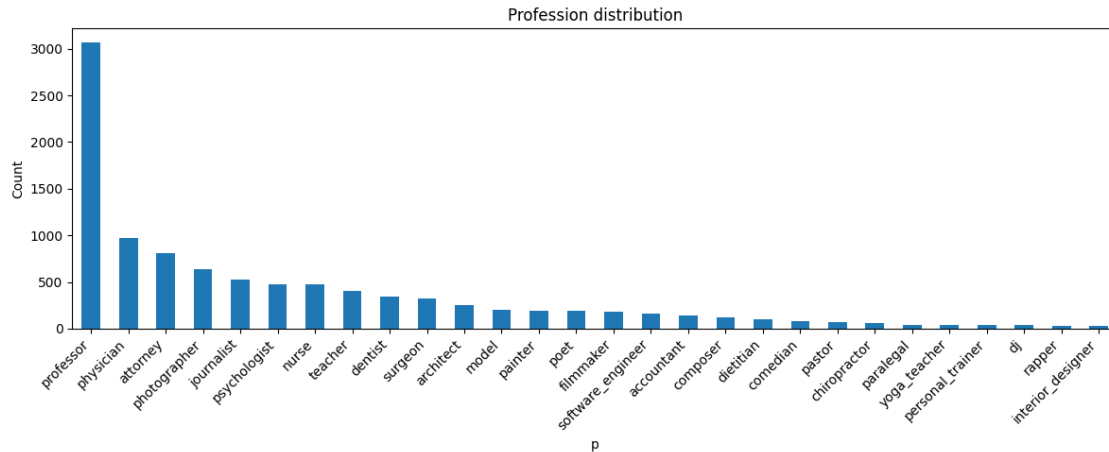
1.3.2 Task 2.2: Verifying minimally-destructive edits.

One key aspect of concept erasure is we want the edits to be *minimal*, i.e., they should not destroy other aspects of the representations. To test the degree to which the representation is preserved, use the profession as a proxy. Show that the profession information is preserved within the embeddings by computing the accuracy (a) before erasure and (b) after erasure.

FOR THE SUBMISSION: Report the accuracy before and after erasure.

```
[ ]: def visualize_profession(df: pd.DataFrame) -> np.ndarray:
    """
    Visualize the label distribution of professions, sorted from highest to
    lowest.
    """
    counts = df["p"].value_counts()
    plt.figure(figsize=(12, 5))
    counts.plot(kind="bar")
    plt.title("Profession distribution")
    plt.ylabel("Count")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()

visualize_profession(df)
```



```
[ ]: def verify_profession_intact(embeddings: np.ndarray, embeddings_erased: np.
    ndarray, df: pd.DataFrame) -> np.ndarray:
    """
    Verify the profession information is preserved within the embeddings.
    """
    professions = LabelEncoder().fit_transform(df["p"])
    acc_before = logreg(embeddings, professions)
    acc_after = logreg(embeddings_erased, professions)
    print(f"Profession accuracy before erasure: {acc_before:.3f}")
    print(f"Profession accuracy after erasure: {acc_after:.3f}")

    verify_profession_intact(embeddings, embeddings_erased, df)
```

Profession accuracy before erasure: 0.926

Profession accuracy after erasure: 0.923

1.3.3 Task 2.3: Show that using a nonlinear classifier, you can recover the concept from the erased embeddings.

While we've successfully removed gender from being *linearly encoded* in the embeddings, they can still be extracted using other models. Your goal is to extract the concept using some other model.

FOR THE SUBMISSION: Report the accuracy before and after erasure.

```
[ ]: from sklearn.model_selection import train_test_split
    from sklearn.neural_network import MLPClassifier

    def nonlinear_classifier(embeddings: np.ndarray, gender_labels: np.ndarray) -> float:
        """
        Train a nonlinear classifier on the embeddings to predict the gender labels
        and return the accuracy.
        """
```

```

"""
X_tr, X_te, y_tr, y_te = train_test_split(
    embeddings,
    gender_labels,
    test_size=0.3,
    random_state=42,
    stratify=gender_labels,
)
clf = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation="relu",
    max_iter=500,
    random_state=42,
)
clf.fit(X_tr, y_tr)
return clf.score(X_te, y_te)

print(
    f"Nonlinear test accuracy before erasure: "
    f"{nonlinear_classifier(embeddings, gender_labels):.3f}"
)
print(
    f"Nonlinear test accuracy after erasure: "
    f"{nonlinear_classifier(embeddings_erased, gender_labels):.3f}"
)

```

Nonlinear test accuracy before erasure: 0.994

Nonlinear test accuracy after erasure: 0.920